

CSCE 641 COMPUTER GRAPHICS · MODULE 01 · LECTURE NOTES

The Graphics Pipeline, End to End

Where a triangle goes, and what happens to it on the way.

MODULE OUTCOMES

Name every stage of the real-time pipeline and state what it consumes and produces.

Explain why rendering is structured as a pipeline at all, and where the parallelism lives.

Trace one triangle from object space to a shaded pixel.

Distinguish fixed-function stages from programmable ones.

Localise a rendering artifact to a stage before opening a debugger.

1 The problem, stated honestly

A renderer is a function. It takes a scene and a camera, and it returns an image. Written that way the whole subject fits on one line, and the line is worth keeping in view, because everything else in this course is an implementation detail of it:

`image = render(scene, camera)`

The difficulty is hidden in the types. A scene is continuous: surfaces are defined everywhere, light varies smoothly, and there is detail at every scale you care to look. An image is a finite grid of samples. Rendering has to cross that boundary, and it has to do it about eight million times per frame at 4K, sixty times a second.

HOLD ON TO THIS

Every stage of the pipeline exists either to make the continuous-to-discrete crossing tractable, or to expose parallelism. If a stage looks arbitrary, you have not yet identified which of the two it is doing.

This framing pays off immediately. Jagged edges, shimmering textures, moiré patterns on a checkerboard floor, and the crawling you see on thin geometry in motion are not four different bugs. They are one phenomenon — undersampling — observed in four places. Module 09 treats it properly; for now, notice that you were told.

2 Two loop orders

There are exactly two sensible ways to organise the computation, and they differ only in which loop is on the outside.

	Ray tracing	Rasterization
Outer loop	Pixels	Triangles
Inner question	Which geometry does this pixel see?	Which pixels does this triangle cover?
Natural strength	Global effects: reflection, refraction, soft shadows, indirect light — anything requiring a secondary ray.	Raw speed and parallelism. Coverage is a cheap geometric test that hardware does extremely well.
Natural weakness	Expense. Every effect costs more rays.	Anything non-local. A triangle does not know what else is in the scene, so reflections and shadows must be faked or precomputed.
Where it is used	Film; offline rendering; increasingly, hybrid real-time.	Real-time: games, visualisation, UI, VR.

This course is rasterization, because rasterization is where the pipeline lives and because you cannot understand a GPU without it. CSCE 647 takes the other loop order seriously and derives light transport properly. They are complementary, not competing — and modern engines run both at once.

3 The six stages

What follows is the canonical real-time pipeline. The names vary between APIs and the hardware fuses stages freely, but the logical structure has been stable for twenty-five years. Learn what crosses each boundary, not just the stage names — the boundaries are what let you localise a bug.

3.1 Application

The CPU side. This stage decides what to draw and in what order, and it is the only stage you control completely. Four jobs matter:

- **Culling.** Discard geometry that cannot contribute. Frustum culling removes what is outside the camera volume; occlusion culling removes what is hidden behind something nearer.
- **Level of detail.** Swap in cheaper meshes when the object is small on screen. A 50,000-triangle model covering nine pixels is pure waste.
- **Sorting.** Opaque geometry front-to-back so the depth test rejects early; transparent geometry back-to-front because blending is order-dependent.
- **Batching.** Each draw call carries fixed overhead. Ten thousand draw calls of one triangle is far slower than one draw call of ten thousand triangles.

WHERE THE WINS ARE

In real engines, the largest performance improvements almost always come from this stage — from *not drawing things* — and not from making shaders faster. The fastest triangle is the one you never submitted.

3.2 Vertex processing

Runs once per vertex, in parallel, on the GPU. Its central job is to move each vertex from the coordinate system it was authored in to the one the rasterizer requires. This is a chain of four spaces:

object → world → camera → clip

Space	Origin is at	Why it exists
Object	The model's own pivot	Artists model one object at a time, without knowing where it will be placed.
World	A chosen scene origin	A common frame so objects can be positioned relative to each other and lit by shared lights.
Camera (view)	The eye, looking down -Z	Makes the camera's position and orientation implicit, so projection need not account for them.
Clip	Homogeneous, pre-divide	The space where the frustum becomes an axis-aligned box, which makes clipping a simple comparison.

Each step is a matrix multiplication, and matrix multiplication is associative, so all three collapse into one matrix per object. The vertex shader therefore does one multiply, not three. Module 03 derives each matrix from scratch.

```
// The entire mathematical content of a basic vertex shader.
out_position = projection * view * model * vec4(in_position, 1.0);

// Note the order. Matrices apply right to left:
// model first (object -> world), then view, then projection.
```

Reversing this order is the single most common beginner error, and it usually produces a black screen rather than a wrong image, which makes it harder to diagnose.

3.3 Clipping and primitive assembly

Vertices are grouped into triangles using the index buffer, and triangles that cross the frustum boundary are cut. Clipping against the side planes is an optimisation; clipping against the *near* plane is a correctness requirement. Geometry behind the eye has negative *w*, and dividing by a negative *w* turns the projection inside out — points behind the camera land in front of it, mirrored. This is why the near plane cannot be zero.

Backface culling also happens here. After projection, compute the signed area of the triangle in screen space; its sign is the winding direction, and one sign means the triangle faces away. On a closed mesh this removes about half of all triangles for the cost of one cross product.

$$2A = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)$$

Signed area of a screen-space triangle. The sign is the winding; the magnitude is twice the area, which Module 04 reuses for barycentric coordinates.

3.4 Rasterization

For each triangle, find the sample points it covers. Each covered sample becomes a **fragment**, carrying interpolated copies of every vertex attribute.

A FRAGMENT IS NOT A PIXEL

A fragment is a *candidate* for a pixel — a proposal that this triangle should determine this pixel's colour.

Many fragments may be generated for one pixel by different triangles. The depth test resolves most of that competition; blending resolves the rest. Keeping the distinction clear prevents real confusion later, especially around multisampling, where a pixel has several coverage samples but may run the fragment shader only once.

Attribute interpolation here must account for perspective. Linear interpolation in screen space is wrong for any attribute that varies linearly in 3D, because projection is not an affine map. Module 04 derives the correction; the symptom when you get it wrong is a texture that appears to slide or swim across a surface as the camera moves, most visible on large floor polygons.

3.5 Fragment processing

Runs once per fragment, and this is where nearly all shader code lives: texture lookups, lighting, material evaluation. The output is a colour and usually a depth.

The cost of this stage scales with **overdraw** — how many fragments are generated per pixel — not with triangle count. Shading a fragment that a nearer fragment will later cover is wasted work, which is the entire justification for front-to-back sorting and for hardware early-Z, where the depth test is moved *before* the fragment shader so that doomed fragments never execute it.

EARLY-Z HAS CONDITIONS

Early-Z is disabled automatically if the fragment shader writes to `gl_FragDepth`, or uses `discard`, because then the hardware cannot know the fragment's depth before running it.

This is a classic silent performance cliff: adding one `discard` for alpha-testing foliage can cost far more than the `discard` itself appears to.

3.6 Output merging

The last stage, fixed-function, and strictly ordered per pixel. Depth test, stencil test, blending, and the write. The ordering guarantee is significant: everything upstream ran out of order across thousands of concurrent threads, yet blending must behave as though triangles were processed in submission order. Hardware spends real resources to maintain that illusion, which is a large part of why blending has never become programmable.

4 Why a pipeline at all

The pipeline structure is not historical accident. Each stage is embarrassingly parallel over its own unit of work — vertices are independent of each other, triangles of each other, fragments of each other — so the hardware can run all six stages simultaneously on different data. While fragments from triangle n are being shaded, triangle $n+1$ is being rasterized and the vertices of $n+2$ are being transformed.

The consequence is that a GPU optimises **throughput**, not latency. A single vertex may take microseconds to traverse the whole pipeline — an eternity — and it does not matter, because millions of other vertices are in flight at the same time. This one commitment explains nearly every architectural feature of GPUs: very wide SIMD, enormous register files, thousands of resident threads, and a willingness to tolerate memory latency that would be catastrophic on a CPU.

THE OPTIMISATION MINDSET THIS IMPLIES

On a CPU you usually make code faster by reducing the work in the critical path.

On a GPU you usually make code faster by *feeding the machine better*: fewer state changes, better batching, higher occupancy, more coherent memory access. The arithmetic is rarely the bottleneck.

CSCE 614 and CSCE 735 make this precise. For now, be suspicious of any graphics optimisation that counts instructions.

5 Reading artifacts

Because each stage has a well-defined input and output, a visual artifact usually implicates one stage. Developing this reflex early will save you more time than any other single habit in this course. Keep the following table somewhere you will actually look at it.

What you see	Stage	Usual cause	First thing to check
Black screen, nothing drawn	Vertex / assembly	Matrix multiplication order reversed, or all triangles culled by winding.	Disable backface culling. If geometry appears, it is winding.
Geometry appears inside-out	Assembly	Negative determinant in the model matrix (mirrored scale) flipping winding.	Check for a negative scale component.
Image stretches when window resizes	Vertex	Aspect ratio not recomputed in the projection matrix.	Is the projection rebuilt on resize?
Texture swims or slides on a floor	Rasterization	Screen-space linear interpolation without perspective correction.	Interpolate attribute/w, then divide by interpolated 1/w.
Two surfaces flicker against each other	Output merging	Z-fighting from insufficient depth precision.	Push the near plane out. Precision loss is dominated by near, not far.
Hard jagged edges, crawling in motion	Rasterization	Point-sampled coverage — aliasing.	Module 09. Enable MSAA to confirm the diagnosis.

What you see	Stage	Usual cause	First thing to check
Transparent objects in wrong order	Output merging	Blending is order-dependent and geometry was not sorted.	Sort transparent draws back-to-front.
Shadows detached from objects	Fragment	Shadow map depth bias too large (peter-panning).	Module 10.

6 The pipeline in twelve lines

Everything above, condensed. Project 1 asks you to implement literally this, so copy it into your repository now and delete comments as you replace them with working code.

```

for each draw call:
    # ---- vertex processing (parallel over vertices)
    for each vertex v:
        clip[v] = P * V * M * v.position

    # ---- assembly, culling, clipping
    for each triangle t:
        if backfacing(t): continue
        for t2 in clip_near(t):

            # ---- perspective divide + viewport
            ndc = t2.clip.xyz / t2.clip.w
            tri = viewport(ndc)

            # ---- rasterization (parallel over fragments)
            for each sample s covered by tri:
                a = perspective_correct_lerp(tri, s)
                color, depth = fragment_shader(a)

            # ---- output merging (ordered)
            if depth_test(s, depth):
                framebuffer[s] = blend(framebuffer[s], color)

```

By the end of Module 05 you will have written every line of this with no graphics API underneath it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lecture 01 — Overview of Computer Graphics	The same framing of what graphics is and why. Watch before this module.

Resource	What it covers for this module
CMU 15-462 Lecture 01 — Course Introduction	Crane's framing of graphics as a sampling-and-representation problem is unusually clear and complements this module directly.
Cem Yuksel — Introduction to Computer Graphics, Lecture 1	The most hardware-grounded overview of the three. Good second pass.
Scratchapixel — 'An Overview of the Rasterization Algorithm'	Written, with code, covering the twelve-line pipeline in detail.
LearnOpenGL — Hello Triangle	Mechanics only: how the stages appear in an actual API. Do not take the theory from here.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Set up your portfolio repository. Add a README containing the twelve-line pipeline from §6, and a checklist of the six stages. You will delete items from it as you implement them.
- 2 Write a program that produces a PNG with no graphics library: allocate a framebuffer, write a gradient, save it. You need this harness for every module through Module 05, and getting it out of the way now means you never again confuse an image-writing bug with a rendering bug.
- 3 Draw a filled triangle into that framebuffer with the crudest method you can think of — brute-force test every pixel against the triangle, however you like. It will be slow and probably slightly wrong along the edges. Keep it; Module 04 replaces it and you will want the comparison.
- 4 For each of the eight artifacts in the diagnosis table of §5, write one sentence predicting what you would check first. Then keep the file — you will revisit your predictions at the end of the course and some of them will be wrong in instructive ways.
- 5 Install RenderDoc. Capture a single frame from any game or application on your machine and find: the draw call count, the depth buffer, and one shader's source. You are not expected to understand the frame yet; the goal is to know the tool exists and opens.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the six stages in order, and for each one state what it consumes and what it produces.
- 2 Why must the near clip plane be greater than zero? What specifically goes wrong at $w = 0$, and what does geometry behind the eye look like if you fail to clip it?

- 3 Explain the difference between a fragment and a pixel, and give a situation where one pixel is associated with many fragments.
- 4 Rasterization and ray tracing differ in loop order. State both loop orders, and give one effect that is natural in each and awkward in the other.
- 5 A GPU targets throughput rather than latency. Give two concrete architectural consequences of that choice, and one consequence for how you should optimise graphics code.
- 6 You see a texture on a large floor plane appear to slide as the camera moves forward. Which stage is at fault and what is the specific cause?